



CODE BASED LEARNING

AMI PREPARED

2026.1

DATABASES – HANDS-ON SQL

PREPARATION

PROF. DR. RER. NAT.
ALEXANDER VOß
FH AACHEN



INTRODUCTION



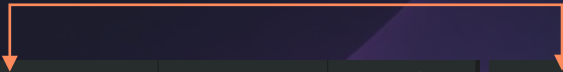
OVERVIEW

What is it all about?

- For the lectures and exercises we will need a real database, or more precisely, a database management system (DBMS). Data is organised in tables (mathematically: relations), which can reference each other via keys.
- This will be used to illustrate the concepts and also to provide the data for the SQL lab and our applications.
- Although there are other systems with different organisational structures, this structured view of the data is sufficient to get you started.

This short tutorial will show you how to

- set this up and
- import the necessary data
- so that we can work with it in the lab.



id	product	category	price	category	name
4	Fish Fingers	1	1.99	1	Frozen Goods
5	Pasta Pan	1	3.29	2	Fruits, Vegetables
6	Carrots	2	0.39	3	Sausages, Cold Meat
7	Onions	2	0.29	4	Milk Products
8	Bananas	2	1.29	5	Cold Drinks
9	Garlic	2	0.98	6	Barbecue Products
10	Pork Sausage	3	1.99	7	Snacks
11	Meatballs	3	2.35	8	Hot Drinks
12	Milk	4	0.79	9	Convenience Foods
13	Quark	4	0.99	10	Baked Goods
14	Yoghurt	4	0.98	11	Magazines
15	Cola light	5	1.50	12	Services

Related product data from the ami_zone schema (with column aliases)



COMPONENTS

We need...

- a language in which to talk to the database → SQL.
- a program for handling commands, results and the database itself → DataGrip.
In principle, however, you can use any other tool or IDE that suits your DBMS.
- a DBMS that provides the data → PostgreSQL.
- a container engine to run the DBMS → Docker or Podman (+ Compose).
- version control for the course material → Git.

```
1
2 ✓ select P.id,P.name "product",C.name "category",P.price "price",P.tax
3      from ami_zone.shop_product P
4      join ami_zone.shop_category C on P.category_id = C.id;
5
6
```

	id	product	category	price	tax
4	4	Fish Fingers	Frozen Goods	1.99	0.07
5	5	Pasta Pan	Frozen Goods	3.29	0.07
6	6	Carrots	Fruits, Vegetables	0.39	0.07
7	7	Onions	Fruits, Vegetables	0.29	0.07
8	8	Bananas	Fruits, Vegetables	1.29	0.07
9	9	Garlic	Fruits, Vegetables	0.98	0.07
10	10	Pork Sausage	Sausages, Cold Meat	1.99	0.07
11	11	Meatballs	Sausages, Cold Meat	2.35	0.07
12	12	Milk	Milk Products	0.79	0.07
13	13	Quark	Milk Products	0.99	0.07
14	14	Yoghurt	Milk Products	0.98	0.07
15	15	Cola light	Cold Drinks	1.50	0.19
16	16	Nerd Bull	Cold Drinks	1.50	0.19

From DataGrip.



NOTES

Although we are mainly working with PostgreSQL, the concepts presented are transferable to other relational DBMSs and, in part, to non-relational (NoSQL) databases.

- PostgreSQL is widely regarded as the most standards-compliant and compatible open-source RDBMS with respect to the standard (currently SQL:2023). All material is written and tested for PostgreSQL 18.
- Note that the SQL dialects of each DBMS are different. The general structure of an SQL command can often be transferred to another DBMS; however, details such as quotation marks or upper and lower case may behave differently. If you want to use the material with another DBMS, you may need to make small adjustments.

The next step is to install your own DBMS while avoiding side effects on your own computer. We do this by running it in a container with Podman or Docker.

- We use Podman or Docker with their respective UIs and Compose to manage the database services. If not installed, get a container engine of your choice.

We use DataGrip by JetBrains (free for students and non-commercial use) to access the DBMS. The installation is usually not critical as it is just an application. Feel free to use your favourite tool, such as pgAdmin.

You can find sample data and also SQL scripts for the SQL lab on GitHub. We assume that you have some knowledge of version control and Git. If not, let us know and we will help you.



CONTAINERISATION



ONE QUESTION FIRST

Is this your own computer and you can install whatever you want?

Yes

Do what you like – but we still recommend containers.
A native PostgreSQL on port 5432 can also clash with existing installs on your own machine.

No

Then that computer may be in production use in some way, and any existing database installations may be affected by another installation.
Use a container.

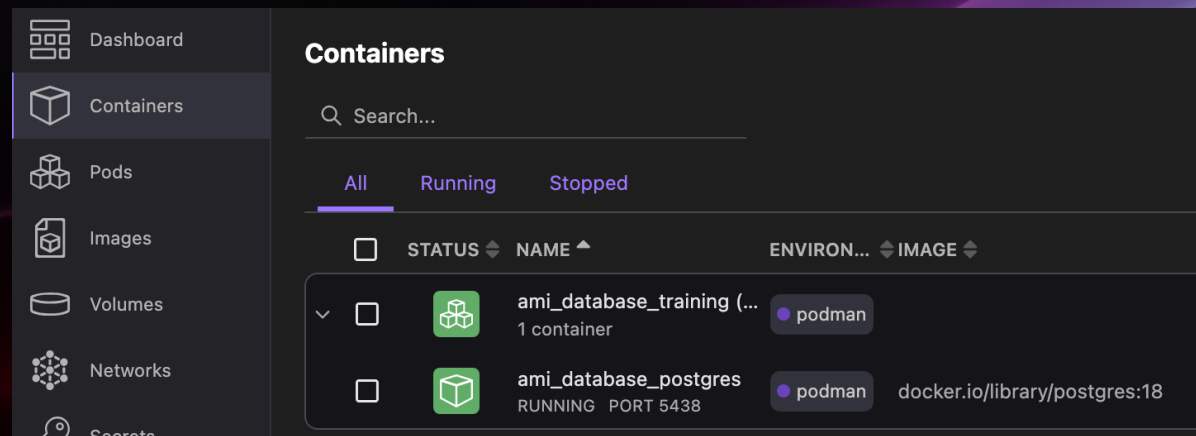




DOCKER OR PODMAN

One way to install a DBMS with as few side effects as possible is to run the database server in a container – managed by a container engine such as Podman or Docker (or similar app).

- We will now show you how to install PostgreSQL in a container.
- To manage the database services, we use Compose and a YAML file describes the service; Compose starts and stops it (see next slide).
- Note: Podman also supports Compose and it works for the majority of use cases, though some advanced features may not be fully compatible.



Podman Desktop.



COMPOSE.YML

```
name: ami_database_training

services:
  postgres:
    # Pinned on purpose: 'latest' would switch to a new major version mid-semester.
    image: postgres:18
    # userns_mode: keep-id # needed for podman on macOS in case of permission errors.
    container_name: ami_database_postgres
    environment:
      POSTGRES_USER: root
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-root}
      # the sample data goes into the default database 'postgres' (otherwise root)
      POSTGRES_DB: postgres
    ports:
      # 127.0.0.1 = reachable from your own machine only, not from the network.
      - "127.0.0.1:5438:5432"
    volumes:
      # PostgreSQL >= 18: mount the parent folder, the data lives in ../18/docker
      - ../postgres_data:/var/lib/postgresql
      # sample data, executed once on first start (alphabetical order)
      - ../preparation:/docker-entrypoint-initdb.d:ro
    restart: unless-stopped
```

Services in compose.yml

Keys

- image – Pinned to version 18.
- environment – Superuser and default database. These are applied only on the very first start; changing them later has no effect.
- ports – host:container. If 5438 is already in use, change only this. 127.0.0.1 means the DB is reachable from your machine only.
- volumes – Containers are disposable: compose down removes them, including everything stored inside. Bind mounts map folders of your computer into the container:
 - postgres_data keeps the data,
 - preparation contains the sample data scripts, executed only when the data directory is empty.
- password – defaults to root; override it in docker/.env if you like.
- userns_mode – Uncomment only if Podman on macOS reports permission errors.



INSTALLATION RECIPE 1/2

Install and start PostgreSQL

- Start a terminal.
- Go to a folder of your choice and clone `database_course` from GitHub.
- Enter the new folder `database_course`.
- Make sure that the configured port is not being used, e.g. by another container.
- Start the DBMS via Compose. Note: the first start pulls the image, so it may take a moment.
- Confirm that the container is running, e.g. via Podman Desktop.
- You will also see that the data directory has been created and later, that initial data is available.

clone the repository:

```
git clone https://github.com/codebasedlearning/database_course.git
```

start a database service (docker or podman)

```
docker compose -f docker/compose.yml up -d
```

stop a service (docker or podman)

```
docker compose -f docker/compose.yml down
```



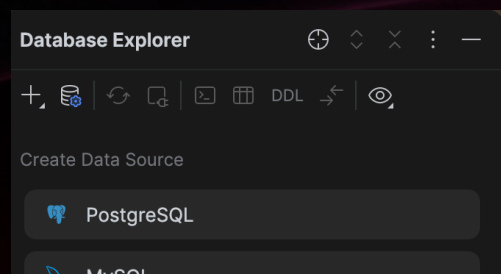
DATA GRIP



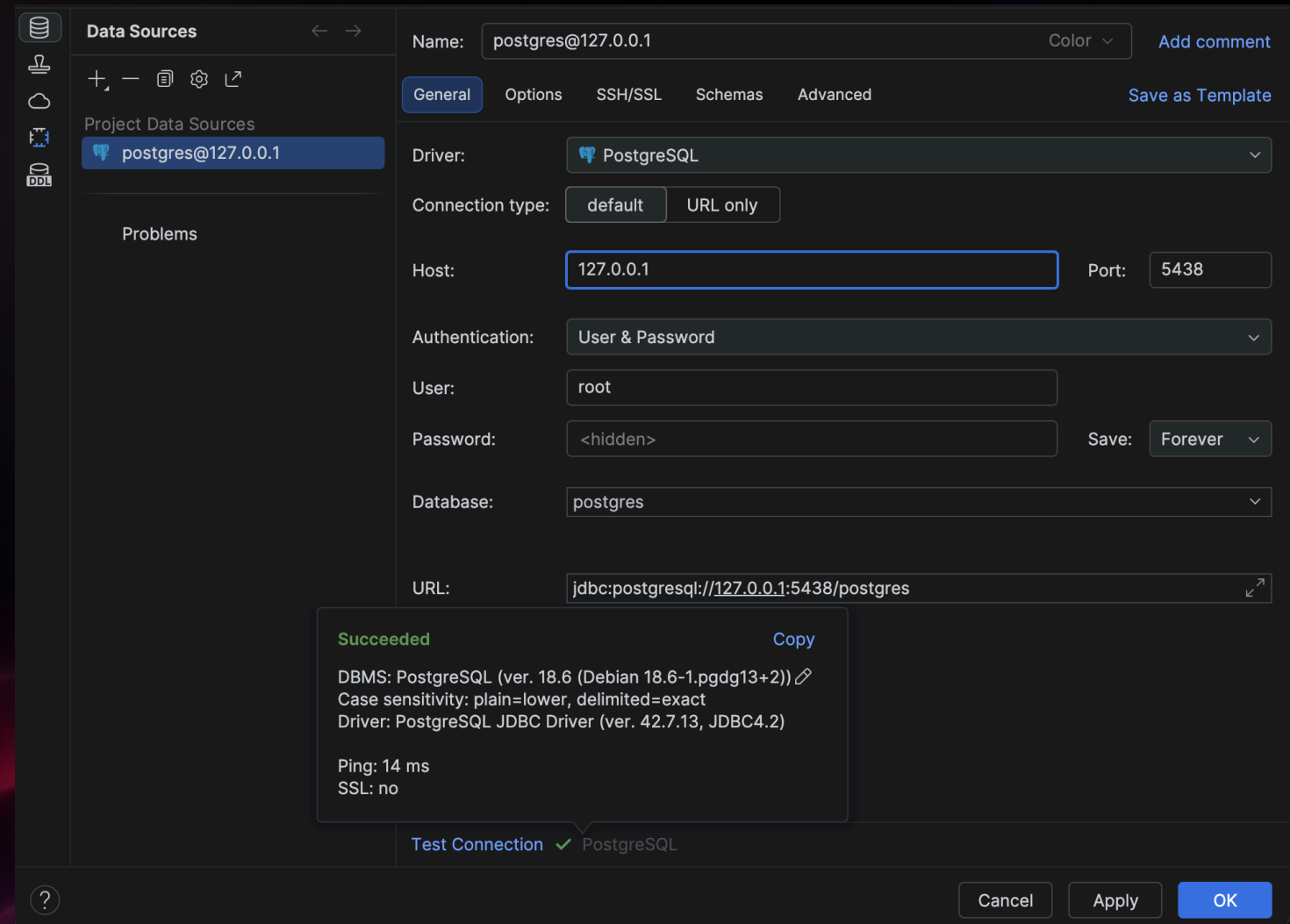
INSTALLATION RECIPE 2/2

Use DataGrip

- Start DataGrip.
- Open Project with folder database_course.
- Create new Data Source type PostgreSQL.
- Enter Name, Host, Port, User, Password and Database. Use 127.0.0.1 as Host, not localhost.
- Test Connection (download missing drivers first, if prompted).



Start screen



Connection Dialog



TABLES

To check that all data was created, open the postgres database and check all ami-schemas (click the 1).

Double-click shop_product from ami_zone. A table with all products should appear.

- If you want to repeat the whole creation process, you can delete the data folder postgres_data, but stop the container first. The next time Compose starts the service, the data is recreated.
Note: new or updated scripts in preparation/ only run on an empty data folder.

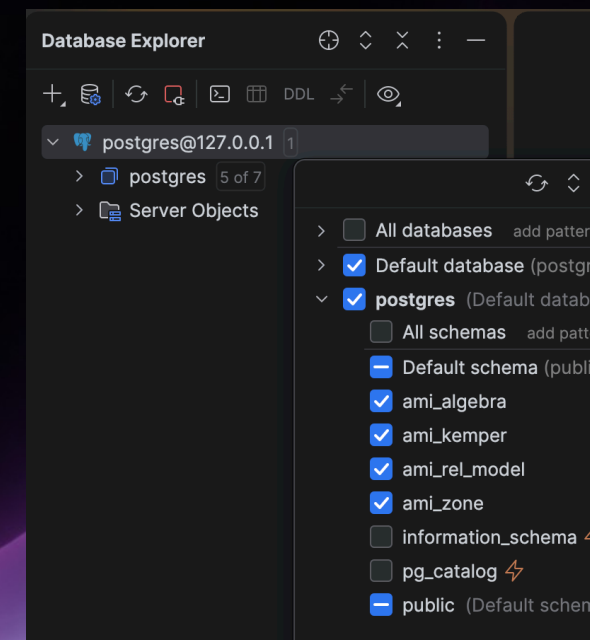


Table select

postgres 5 of 7		id	name
ami_algebra	1	1	Spinach
ami_kemper	2	2	Four Cheese Pizza
ami_rel_model	3	3	Spinach Pizza
ami_zone	4	4	Fish Fingers
asset_catalogue	5	5	Pasta Pan
asset_inventory	6	6	Carrots

Table results



CONSOLE

Open a new Query Console and run your first SQL statement:

```
select * from ami_zone.shop_product;
```

and run it with Ctrl/Cmd + Enter. The execution of a single command or script takes place in a session belonging to a data source/connection. You can also change this session if you have more than one data source.

The screenshot shows the DataGrip Query Console interface. At the top, there's a toolbar with icons for running queries, saving, and other functions. Below the toolbar, the SQL statement `select * from ami_zone.shop_product;` is entered in the editor, and it has been executed successfully, as indicated by a green checkmark and the text "423 ms". Below the editor, the "Output" tab is selected, showing the results of the query in a table format. The table has columns for id, name, category, unit, and price. The results are as follows:

id	name	category	unit	price
1	Spinach		1 PK	
2	Four Cheese Pizza		1 PC	
3	Spinach Pizza		1 PC	

Console



LINKS



LINKS

Podman <https://podman.io>

Podman Desktop <https://podman-desktop.io>

Podman install guide <https://podman.io/docs/installation>

Docker <https://docs.docker.com/get-started/get-docker/>

DataGrip <https://www.jetbrains.com/datagrip/>

Git <https://git-scm.com/install/>

GitHub https://github.com/codebasedlearning/database_course

PostgreSQL 18 docs <https://www.postgresql.org/docs/18/>

postgres image (Docker Hub) https://hub.docker.com/_/postgres